Rheinisch-Westfälische Technische Hochschule Aachen
Alessandro Berti

Phase Review: 2nd Sprint of the Implementation of "Conformance Predictions"

Group 4:
Richard Schuppener, Cem Colak

Contents

1 Summary of Sprint 2 **3**

2 Codebase Structure **4**

3 Testing **5**

4 Requirements Coverage Status - Overview per Module **5**
4.1 IO 5
4.2 Feature Extraction 6
4.3 Model Training 7
4.4 Evaluation 7
4.5 Config & Pipeline 8
4.6 Project Infrastructure & Testing 9

5 User Guide **10**

6 Team settings and Workflow **10**
6.1 Changes in Responsibilities 10
6.2 Workflow Improvements for the next sprint 11

7 Exemplary Visualizations and Outputs **11**

8 Sprint review **11**
8.1 Richard Schuppener 11
8.2 Cem Colak 11

1 Summary of Sprint 2

In this sprint, we focused on implementing all the remaining functionalities required for a fully functional end-to-end pipeline for conformance predictions. We intend to use the final sprint to further refine and optimise these functionalities, so that we can deliver a well-aligned and polished software artifact that fulfils Alessandro's requirements in the most sophisticated way.

Key achievements of this sprint include:

1. Full end-to-end support for all three prediction targets – *Future Conformance Deviations*, *Process Outcome*, and *Remaining Time* – with the CLI, pipeline, feature extraction, and model training extended accordingly.

2. Introduction of a prefix-generation module (`feature_extraction.generate_prefixes`) supporting both *exactly-k* and *varying* $1..k$ prefix modes, with outcome-leakage gating that prevents the target activity from ever appearing inside the feature vector.

3. Implementation of the *Case Attributes* and *Hybrid* feature paths, including robust detection of trace-level attributes for both prefixed (`case:`) and unprefixed (XES-style) columns.

4. A configurable deviation cost threshold T for the Future Deviations target, backed by a corrected cost function that excludes silent (τ) routing moves from the real deviation count.

5. Per-prefix *remaining-time* regression labels and dedicated temporal predictors (elapsed time, time since last event, event count).

6. Extension of the ML model menu beyond the Decision Tree: *Logistic Regression* and *Gradient Boosting* for classification, and *Random Forest*, *Gradient Boosting*, and *Decision Tree Regressor* for regression.

7. Runtime and memory instrumentation via `time.perf_counter` and `psutil`, with optional persistence of the collected measurements as a structured JSON report.

8. Prefix-length performance analysis (`metrics_by_prefix_length`) that reports the full metric set per prefix length for a single pooled model.

9. Pytest suite covering feature extraction (case attributes, prefix gating, remaining time, deviation cost), model training (parametrised over all classifier and regressor variants), evaluation metrics, and IO.

10. Extraction of a dedicated *Visualization* module that bundles process-model, decision-tree, confusion-matrix, and per-class accuracy rendering.

2 Codebase Structure

We enriched the initial codebase structure outlined in the first sprint review with a dedicated visualization module that bundles all visualization functionalities together. The modules are decomposed into the following categories:

1. **IO:** Handles inputs and outputs, such as reading event logs and normative models, as well as visualizing process models.

2. **Feature Extraction:** Performs conformance checking (alignments only, hence no separate conformance module), extracts features related to alignments, control flow, and case attributes, and preprocesses the data for ML training (e.g. scaling of numerical values).

3. **Model Training:** Trains a model (as specified by the user) on the target and feature vector produced by the Feature Extraction step. This module also provides the capability to predict unseen data (test data or new data).

4. **Evaluation:** Computes evaluation metrics such as accuracy, precision, recall/fitness, confusion matrix, and ROC.

5. **Config:** Collection of Enums that correspond to the options presented to the user via the CLI. It keeps the codebase clean and easier to manage.

6. **Pipeline:** Bundles functionalities from the other modules into the larger "steps" of the process. Based on the user's choices, the correct functions are called and their inputs and outputs are streamlined together.

7. **Main / CLI:** The entry point of the program; also serves as the guide and option selector for the user. The CLI presents options, numerical metrics, and comprehensive error messages.

8. **Visualization:** Bundles all visualization functionalities, allowing us to maintain and adjust the visualizations in a more modular manner.

3 Testing

4 Requirements Coverage Status - Overview per Module

4.1 IO

All Covered

- **LL-01.1**: `io.discover_process_model()` supports Inductive, Alpha, and Genetic Miner via PM4Py, enabling automatic model discovery when no normative model is available.
- **LL-01.2**: `io.visualize_process_model()` renders the model as Petri net, BPMN, or Process Tree, with optional file saving or direct display.
- **HL-05**: Event log loading, model import, and model discovery are all functional; the foundational data-acquisition step of the pipeline is complete.
- **LL-05.1**: `pm4py.read_xes` for XES logs; `pd.read_csv` combined with `pm4py.format_dataframe` for CSV logs in `io.load_event_log()`.
- **LL-05.2** (partially): Petri net loading via `pm4py.read_pnml` implemented; BPMN files loaded via `pm4py.read_bpmn` and Process Tree files via `pm4py.read_ptml`, both internally converted to a Petri net.
- **LL-05.3**: Inductive, Alpha, and Genetic Miner available as fallback discovery algorithms when no normative model file is provided.
- **HL-04**: `try/except` blocks in `main.py` catch and report setup errors, preprocessing failures, and pipeline exceptions to `stderr` with descriptive labels.
- **HL-01**: End-to-end prediction of conformance deviations is now fully operational; the *Future Deviations* target is wired through the complete pipeline – label extraction via `real_deviation_cost`, configurable threshold T, classification training, and evaluation.
- **LL-05.2**: Standalone loading of BPMN and Process Tree files as direct model inputs (without forced internal conversion to a Petri net).

4.2 Feature Extraction

Covered

- **HL-06** (partially): `feature_extraction.compute_alignments()` invokes `pm4py.conformance_diagnostics_alignments`; log-moves, model-moves, and synchronous moves are extracted per case. Distance-to-completion indicators are not yet included.
- **LL-06.1**: PM4Py alignment dictionaries are computed for every trace against the normative Petri net using the standard cost function. Alignments are now keyed by case ID and re-emitted in first-appearance order, eliminating the silent ordering issue that previously misaligned diagnostics with cases.
- **LL-06.2**: Alignment moves are parsed by `real_deviation_cost()` to count genuine deviations (log-moves and visible model-moves) per case; cumulative alignment cost is extracted while silent (τ) routing moves are correctly excluded.
- **HL-07**: `feature_extraction.extract_features()` now supports control-flow features (Bag-of-Activities), alignment-based diagnostic features (Bag-of-move-types per activity), case-attribute features, and the hybrid combination of all three.
- **LL-07.3**: Control-flow frequency matrix constructed via `pd.crosstab` over case IDs and activity names, producing a per-case Bag-of-Activities feature matrix.
- **HL-13**: Alignment results are computed once per run and stored in memory; they are passed as a variable to avoid redundant re-computation within a single pipeline execution.
- **HL-02**: Structured case attributes are fully integrated as predictive features. `list_case_attributes` and `summarize_case_attributes` detect both explicitly prefixed (`case:`) and constant-within-case attributes (XES-style), and the CLI lets the user select a subset of them.
- **LL-07.1**: `generate_prefixes()` expands each case into varying-length prefix pseudo-cases, with optional outcome-leakage gating, supporting both *exactly-k* and *varying* $1..k$ modes.

- **LL-07.2**: `extract_remaining_time()` and `extract_temporal_features()` together produce a numerical feature matrix tailored to the remaining-time regression target, including elapsed time, time since the previous event, and event count.
- **HL-06**: *We left this out.* Addition of distance-to-completion indicators (e.g. remaining enabled transitions, token distance to the final marking).

Planned in Sprint 3

- **LL-13.1**: *Optional:*Hash-based caching of prefix activity sequences to deterministically identify and skip redundant alignment computations.
- **LL-13.2**: *Optional:* Disk-based persistence of alignment results via `joblib`.

4.3 Model Training

Covered

- **HL-08**: `train_classification_model()` wraps a configurable classifier (Decision Tree, Logistic Regression, or Gradient Boosting) in a `sklearn.Pipeline`. The binary deviation label is derived from the case's real alignment cost via the configurable threshold T, so the target is fully driven by alignment-based deviation cost.
- **LL-08.1**: `StandardScaler` is integrated as the first step in every sklearn Pipeline, ensuring scale-consistent feature distributions.
- **HL-09**: Baseline (control-flow features only) and conformance-aware (diagnostics or hybrid features) model variants are both supported via the `PredictionFeature` configuration.
- **LL-08.2**: Additional ML algorithms are available beyond the Decision Tree: *Logistic Regression* and *Gradient Boosting* for classification, and *Random Forest*, *Gradient Boosting*, and *Decision Tree Regressor* for regression. All are exposed via the CLI and selectable per run.

4.4 Evaluation

Covered

- **HL-15** (partially): The main pipeline uses `pm4py.split_train_test` with a user-configurable train ratio (default 0.7); a strict temporal train/validation/test split is not yet enforced.
- **HL-16** (partially): `evaluation.compute_classification_metrics()` computes accuracy, precision, recall, F1-score, ROC-AUC, and Brier score for binary and multi-class classification targets.
- **HL-12**: Confusion matrix heatmap and per-class accuracy bar chart are generated via Matplotlib in `visualization.save_evaluation_visuals()`.
- **HL-17**: `evaluation.metrics_by_prefix_length()` slices the predictions of a single pooled model by prefix length and reports the full metric set per length; the CLI prints this table whenever prefix lengths vary.
- **LL-16.2**: `compute_mae`, `compute_mse`, and `compute_rmse` are available in the evaluation module for evaluating remaining-time regression targets.

Planned in Sprint 3

- **HL-15**: Full integration of a strict temporal split (train, validation, test) ordered by case end time into the main pipeline.
- **HL-16**: Conformance quality metrics for the normative model (fitness), and wiring of the regression metrics into the pipeline's `compute_regression_metrics` hook so they are reported automatically for the remaining-time target.

4.5 Config & Pipeline

Covered

- **HL-03**: `main.py` provides an interactive CLI that guides the user through model path/discovery selection, prediction target (future deviations, process outcome, remaining time), feature basis (including case attributes), prefix mode, train/test split ratio, and ML algorithm configuration.
- **LL-03.1**: Prediction target and feature basis options are fully defined in `config.py` and selectable in the CLI; both the case-attribute feature path and the conformance-deviation target are now fully wired through the pipeline.

- **HL-10**: `main.py` combined with `pipeline.py` forms an end-to-end runner from raw data and model input to evaluation output, without manual intermediate steps.
- **HL-14**: Runtime and resource measurement now wrap conformance checking, feature extraction, model training, and evaluation; the totals are reported at the end of every run.
- **LL-14.1**: `evaluation.measure_runtime()` uses `time.perf_counter()` around alignment computation, feature extraction, model training, and evaluation.
- **LL-14.2**: `evaluation.measure_memory()` polls `psutil.Process().memory_info()` to track peak RSS during the entire pipeline run.
- **LL-14.3**: Sub-second precision wall-clock measurements are recorded per step (conformance checking, feature generation, training, evaluation).
- **LL-14.4**: Runtime and resource metrics are printed to the console and can be persisted to a structured JSON log file at the end of the run.

4.6 Project Infrastructure & Testing

Covered

- **HL-18**: PM4Py, scikit-learn, pandas, NumPy, and Matplotlib are used as the primary library stack throughout all modules.
- **HL-19 & LL-19.1-19.2**: The project is organized into `io`, `feature_extraction`, `model_training`, `evaluation`, `pipeline`, `visualization`, and a centralized `config` module; Git is used for version control. Note: the conformance logic is consolidated into `feature_extraction` rather than placed in a dedicated `conformance` module. A formal branching strategy per module was not adopted, as we are working with one branch per team member.
- **HL-20**: `requirements.txt` and `README.md` are present in the repository.
- **LL-20.1**: `requirements.txt` as a plain text file is implemented and considered satisfactory for dependency management.

- **HL-11**: An automated pytest test suite covering IO, feature extraction, model training, and evaluation is executable via a single command (`pytest tests/`) and runs in the GitLab CI pipeline on every push.
- **LL-11.3**: `test_generate_prefixes_lengths_and_content` asserts that a prefix of length k contains exactly the first k events of the original trace; the related `test_pooled_extraction_broadcasts_labels_without_leakage` test additionally confirms that outcome activities are gated out of the feature vector.

Planned in Sprint 3

- **LL-11.1**: Dedicated pytest fixtures providing a synthetic, fully conforming XES log and a matching Petri net (currently the tests use ad-hoc DataFrame helpers).
- **LL-11.2**: Explicit test assertion that the alignment cost equals 0.0 on a fully conforming synthetic dataset.
- **HL-21**: Technical documentation of system architecture.

5 User Guide

In this sprint, we did not update the existing User Guide, as we focused heavily on the implementation and considered the CLI sufficient for testing from a reviewer's perspective. We plan to write a comprehensive User Guide in Sprint 3, so that a user with no prior process mining experience can operate, understand, and interpret the tool's functionalities.

6 Team settings and Workflow

6.1 Changes in Responsibilities

As already stated in the first sprint review, we rotated responsibilities for this sprint. Cem was responsible for IO, Model Training, and Feature Extraction, while Richard was responsible for the new visualization module, evaluation, and pipeline. Both of us were jointly responsible for the CLI and config modules, as they are relatively small and we wanted to work on them together to maintain a shared understanding of the user interface and configuration options. This division of responsibilities worked well, as it allowed each of us to understand and reflect on the other's past contributions and to extend them with new capabilities.

For the next sprint, we plan to keep the same split on the implementation side and place a stronger focus on documentation. We intend to work together on the User Guide, the technical documentation, the README, and the phase review. This will allow us to review each other's content under the four-eyes principle, ensuring more robust output quality.

6.2 Workflow Improvements for the next sprint

In this sprint, we revised our branching and workflow strategy. First, we adopted a more granular decomposition of the work items in Git, which allowed us to precisely track and address the planned requirements for this sprint. Second, we consolidated our personal branches into three key branches – `development`, `documents`, and `main` – and created dedicated branches per work item, which were later merged into their respective parent branches. We will continue with this approach, as it has worked very well so far.

For the next sprint, we plan to adopt an even more issue-based workflow on the code side and a more granular approach to both the creation and the content of the documents. As this is the final sprint, all documents are also intended to be completed within the deadline.

7 Exemplary Visualizations and Outputs

8 Sprint review

8.1 Richard Schuppener

review

8.2 Cem Colak

In this sprint we switched the responsibilities. This was a very important step for me, to really understand the other modules of our project and add/improve improve some functionalities. I feel very comfortable with our codebase, and even see some potential implementation improvements we can make in the next sprint. Furthermore, the new branching and granularity was pretty new for me, although that I know, that it is common pratice. I really liked the approach and will definitely use it in my future projects as well. Additionally, I noticed that preprocessing is a pretty heavy module and understand now, why Prof. Van der Aalst meant, that Preprocessing makes a huge stake in the whole data pipeline. In terms of time-management, I was

a bit restricted in the second week, since I am on vacation. Still, Richard and me already planned and timed our engagement in the whole sprint such that we were able to manage all our goals for the second sprint in a smooth and timely manner. The only thing I didn't expect is the weak level of documentation we have had so far. On the other hand, I was really busy in the past days with my seminar submission and all the assignments, so I think it is a valid reason why we achieved less in this part of the project. For the next sprint, we plan to focus on the documentation from the beginning, as we have alrady implemented most of the functionalities and can produce a valuable documentation base. In terms of the collaboration with Richard I'm very satisfied overall and valued his engagement in planning and adivsing a lot. We have had regular conversations and calls such that our boths views are aligned and no one works apart from the other. I'm looking foward continuing and finishing the project with Richard in the incoming last sprint.

review